
État d'Avancement du Projet

NEXORA

*Plateforme d'Intelligence Opérationnelle pour la Gestion
et la Valorisation des Compétences Internes*



Réalisé par :

Soubaili Omar Yassine Baizi

Encadré par :

Mme. OUNACER SOUMAYA

Filière : 4IADM G5-A

Année Universitaire : 2025-2026

Date du rapport : 16 avril 2026

Table des matières

1	Introduction	2
1.1	Objectifs du rapport	2
2	Rappel du Planning Prévisionnel	2
3	Avancement Global par Phase	3
4	Phase 1 — Analyse & Conception	3
4.1	État : Complète (100%)	3
4.1.1	Documents de Conception (LaTeX)	3
4.1.2	Livrables PDF compilés	3
4.1.3	Diagrammes UML et Schémas Générés	4
4.1.4	Maquettes UI/UX	4
5	Phase 2 — Développement Backend	4
5.1	État : Complète (100%)	4
5.1.1	Application FastAPI — Infrastructure Core	5
5.1.2	Modules de Routage API (18 routers — 5 532 lignes)	5
5.1.3	Moteur d'Intelligence Artificielle (9 modules ML — 1 835 lignes)	5
5.1.4	Détail des Algorithmes Implémentés	6
5.1.5	Big Data Pipeline (6 modules Spark)	6
5.1.6	Endpoints API REST — Inventaire Complet	6
5.1.7	Données de Test et Jeux de Données	8
6	Phase 3 — Développement Frontend	8
6.1	État : Complète (100%)	8
6.1.1	Pages de l'Application (18 pages — 7 068 lignes)	9
6.1.2	Architecture Frontend	9
6.1.3	Stack Technique Frontend	10
7	Phase 4 — Intégration & Tests	10
7.1	État : En cours (80%)	10
7.1.1	Infrastructure de Tests	10
7.1.2	Détail de la Couverture par Module	10
7.1.3	Actions Restantes pour la Phase 4	11
8	Phase 5 — Déploiement & Livraison	11
8.1	État : Complète (100%)	11
8.1.1	Infrastructure Docker	12
8.1.2	Documentation du Projet	12
9	Modèle de Données Neo4j	12
9.1	Nœuds	12
9.2	Relations	13
10	Statut du Projet Comparé au Diagramme de Gantt	13
10.1	Analyse des Écarts	13

11 Métriques Quantitatives du Projet	13
11.1 Tableau de bord récapitulatif	14
11.2 Métriques de Code Source	15
11.3 Stack Technologique Complète	16
12 Plan d'Action Restant	16
13 Conclusion	16

1. Introduction

Ce document constitue le rapport d'avancement exhaustif du projet **Nexora**, une plateforme d'intelligence opérationnelle conçue pour la gestion et la valorisation des compétences internes au sein d'une organisation. Le système combine des techniques d'**Intelligence Artificielle** (PageRank, filtrage collaboratif, TF-IDF, LLM), du **Big Data** (PySpark, simulation Kafka), et une **base de données orientée graphe** (Neo4j) pour offrir une cartographie intelligente des expertises.

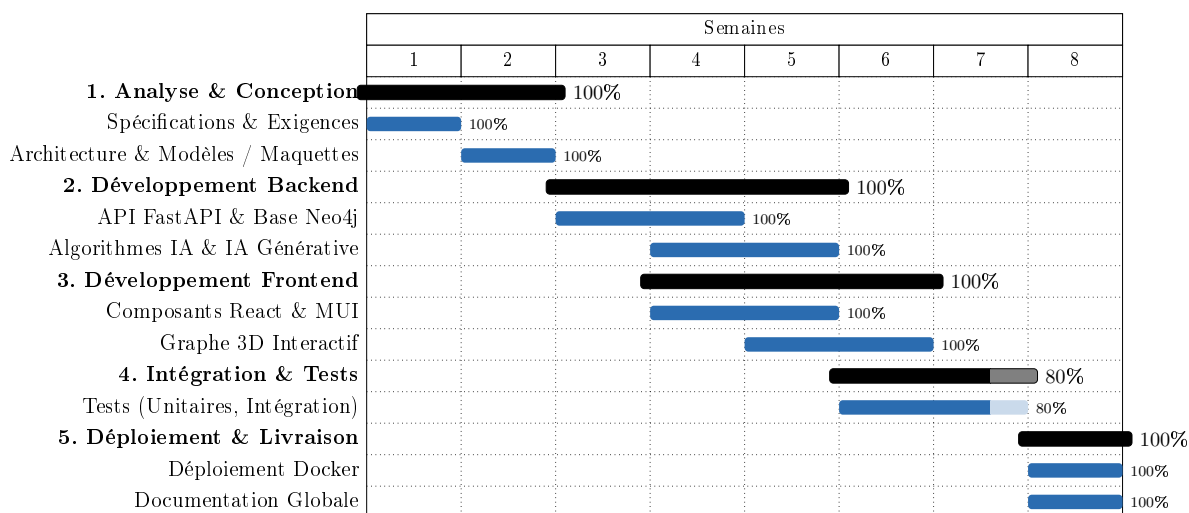
Le projet est planifié sur **8 semaines** et suit une méthodologie de développement itérative avec des phases parallélisées de conception, développement, tests et déploiement.

1.1 Objectifs du rapport

- Fournir un inventaire complet et détaillé de tous les composants réalisés
- Présenter les métriques quantitatives du projet (lignes de code, fichiers, couverture)
- Analyser l'écart entre le planning prévisionnel et l'avancement réel
- Identifier les travaux restants et proposer un plan de finalisation

2. Rappel du Planning Prévisionnel

Le diagramme de Gantt ci-dessous rappelle la planification initiale du projet, avec l'état d'avancement de chaque tâche.



Légende : Barres pleines = tâches complétées | Barres claires = tâches en cours ou partielles

3. Avancement Global par Phase

Phase	Semaines	Avancement	Statut
1. Analyse & Conception	S1 – S2	100%	✓ Complète
2. Développement Backend	S3 – S5	100%	✓ Complète
3. Développement Frontend	S4 – S6	100%	✓ Complète
4. Intégration & Tests	S6 – S7	80%	▲ En cours
5. Déploiement & Livraison	S8	100%	✓ Complète

4. Phase 1 — Analyse & Conception

4.1 État : Complète (100%)

L'ensemble des livrables de conception ont été rédigés, compilés en PDF et validés. La documentation couvre tous les aspects du système, depuis le cahier des charges jusqu'aux maquettes UI.

4.1.1 Documents de Conception (LaTeX)

Livrable	Statut	Taille	Fichier source
Cahier des Charges complet	✓	23.8 Ko	Cahier_des_Charges_Nexora.tex
Étude comparative des solutions	✓	17.8 Ko	Chapitre2_Etude_Comparative.tex
Description des datasets	✓	23.0 Ko	Chapitre3_Description_Datasets.tex
Architecture globale du système	✓	30.0 Ko	Chapitre4_Architecture_Globale.tex
Conception UML détaillée	✓	33.2 Ko	Conception_Nexora.tex
Maquettes UI/UX	✓	32.0 Ko	Maquettes_Nexora.tex

4.1.2 Livrables PDF compilés

Tous les documents LaTeX ont été compilés en PDF finalisés, disponibles dans le répertoire `Livrables/` :

PDF	Statut	Taille
Cahier des Charges.pdf	✓	192 Ko
Chapitre2_Etude_Comparative.pdf	✓	228 Ko
Chapitre3_Description_Datasets.pdf	✓	304 Ko
Chapitre4_Architecture_Globale.pdf	✓	289 Ko
Chapitre_Conception.pdf	✓	1.25 Mo
Maquette.pdf	✓	2.43 Mo

4.1.3 Diagrammes UML et Schémas Générés

Diagramme	Statut	Taille
Diagramme de cas d'utilisation	✓	333 Ko
Diagramme de classes	✓	213 Ko
Séquence : Authentification	✓	111 Ko
Séquence : PageRank	✓	108 Ko
Séquence : Recommandation	✓	108 Ko
Schéma du graphe Neo4j	✓	109 Ko
Architecture globale	✓	133 Ko

Le script `generate_diagrams.py` (28.9 Ko) permet la génération automatique de l'ensemble des diagrammes.

4.1.4 Maquettes UI/UX

Maquette	Statut
Page de connexion (Login)	✓
Tableau de bord (Dashboard)	✓
Graphe 3D interactif	✓
Insights IA	✓
Évolution des compétences	✓

5. Phase 2 — Développement Backend

5.1 État : Complète (100%)

Le backend FastAPI est intégralement développé avec **18 modules de routage** (5 532 lignes de code), un moteur ML complet (9 modules, 1 835 lignes), et l'infrastructure Big Data (6 modules Spark). L'ensemble du backend comprend **40 fichiers source** pour un total de **~386 Ko** de code Python.

5.1.1 Application FastAPI — Infrastructure Core

Composant	Statut	Taille	Description
app/main.py	✓	3.4 Ko	Point d'entrée FastAPI, 18 routers
app/config.py	✓	1.5 Ko	Configuration Pydantic Settings
app/database.py	✓	2.0 Ko	Driver Neo4j (singleton)
app/auth_utils.py	✓	1.5 Ko	JWT + bcrypt + OAuth2
app/fallback_data.py	✓	8.2 Ko	Données de repli (mode hors-ligne)
app/ws_manager.py	✓	2.3 Ko	WebSocket Manager temps réel
.env + .env.example	✓	—	Variables d'environnement
requirements.txt	✓	355 o	15 dépendances Python
users.json	✓	2.2 Ko	Base utilisateurs locale (dev)

5.1.2 Modules de Routage API (18 routers — 5 532 lignes)

Router	St.	Taille	Router	St.	Taille
auth.py	✓	10.6 Ko	notifications.py	✓	3.8 Ko
experts.py	✓	10.9 Ko	skillmap.py	✓	13.0 Ko
documents.py	✓	12.7 Ko	workspaces.py	✓	19.5 Ko
graph.py	✓	12.8 Ko	gamification.py	✓	9.1 Ko
dashboard.py	✓	9.4 Ko	network.py	✓	5.8 Ko
ai.py	✓	18.0 Ko	kafka_simulator.py	✓	10.1 Ko
bigdata.py	✓	13.3 Ko	learning.py	✓	24.8 Ko
feed.py	✓	14.0 Ko	learning_resources.py	✓	67.3 Ko
jobs.py	✓	19.0 Ko	messaging.py	✓	9.8 Ko

5.1.3 Moteur d'Intelligence Artificielle (9 modules ML — 1 835 lignes)

Module ML	Statut	Taille	Algorithme / Technique
pagerank.py	✓	8.4 Ko	Analyse topologique PageRank (damping=0.85)
recommender.py	✓	11.5 Ko	Filtrage collaboratif + similarité cosinus
skill_predictor.py	✓	20.0 Ko	Prédiction compétences émergentes + forecasting
classifier.py	✓	6.4 Ko	TF-IDF + Régression Logistique
chatbot.py	✓	22.2 Ko	LLM Groq + fallback rule-based (Veda)
embeddings.py	✓	4.1 Ko	Vectorisation sémantique TF-IDF
graph_rag.py	✓	4.3 Ko	Retrieval-Augmented Generation sur graphe
summarizer.py	✓	2.0 Ko	Résumé automatique de documents (LLM)
analytics.py	✓	6.4 Ko	Analyses statistiques et prédictives

5.1.4 Détail des Algorithmes Implémentés

1. **PageRank Expert Scoring** : Construction d'un graphe bipartite Expert-Skill-Project, exécution itérative du PageRank (damping=0.85, 20 itérations). Score final : pondération graphe (60%) + features de base (40%) : diversité de compétences, profondeur, participation aux projets, documents publiés, ancienneté.
2. **Filtrage Collaboratif (Skill Gap)** : Matrice utilisateur-compétence avec similarité cosinus. Identification des 10 experts les plus similaires, recommandation de leurs compétences manquantes pondérées par similarité $\times$ niveau.
3. **Détection de Compétences Émergentes** : Analyse par cohortes temporelles (date d'embauche). Comparaison des taux d'adoption entre employés récents (<2 ans) et anciens (2–5 ans). Seuil d'émergence : croissance $>20\%$.
4. **Prévision de Demande Future** : Combinaison demande projet-compétence + taux de croissance émergent pour projeter la demande à N mois. Identification des pénuries critiques.
5. **Collaboration Inter-Départements** : Analyse du chevauchement et de la complémentarité des compétences entre départements. Scoring des paires à haute complémentarité.
6. **Classification de Documents (TF-IDF)** : Vectorisation TF-IDF + classification par Régression Logistique multi-classes.
7. **Chatbot Veda** : Intégration API Groq (LLM) avec fallback rule-based. Support de requêtes sur les experts, compétences, projets.
8. **Graph RAG** : Conversion langage naturel $\rightarrow$ requêtes Cypher pour interrogation du graphe Neo4j.

5.1.5 Big Data Pipeline (6 modules Spark)

Composant	Statut	Taille	Description
spark_batch.py	✓	11.7 Ko	Moteur de traitement batch PySpark
skill_analytics.py	✓	5.4 Ko	Analyses de compétences à grande échelle
expert_scoring.py	✓	5.7 Ko	Scoring d'experts distribué
document_processing.py	✓	5.6 Ko	Traitement de documents en batch
company_analytics.py	✓	9.7 Ko	Analyses départementales et projets
generate_all.py	✓	17.6 Ko	Générateur complet (fallback Python)
kafka_simulator.py	✓	10.1 Ko	Simulation pipeline Kafka temps réel

5.1.6 Endpoints API REST — Inventaire Complet

L'API expose un total de **>50 endpoints** répartis dans les catégories suivantes :

Méthode	Endpoint	Description
Authentification & RBAC		
POST	/api/auth/register	Inscription nouvel utilisateur
POST	/api/auth/login	Connexion (retourne JWT)
GET	/api/auth/me	Profil utilisateur courant
GET	/api/auth/users	Liste utilisateurs (admin)
PUT	/api/auth/users/{name}/role	Modification rôle (admin)
Intelligence Artificielle		
POST	/api/ai/chatbot	Assistant IA Veda
POST	/api/ai/recommend-experts	Recommandation ML d'experts
GET	/api/ai/expert-rank	Scoring PageRank
GET	/api/ai/emerging-skills	Compétences émergentes
GET	/api/ai/future-skills	Prévision demande future
GET	/api/ai/cross-department-suggestions	Collaboration inter-dpt.
GET	/api/ai/personalized-recommendations/{id}	Recommandations personnalisées
GET	/api/ai/skill-gaps/{id}	Analyse des lacunes
POST	/api/ai/classify-document	Classification automatique
Big Data		
GET	/api/bigdata/skill-analytics	Analytics Spark
GET	/api/bigdata/expert-rankings	Rankings d'experts
POST	/api/pipeline/simulate	Simulation pipeline Kafka
GET	/api/pipeline/status	État du pipeline
Données & Recherche		
GET	/api/experts/search	Recherche multicritères d'experts
GET	/api/documents/search	Recherche de documents
GET	/api/graph/nodes	Nœuds du graphe
GET	/api/dashboard/stats	Statistiques globales
Collaboration & Social		
GET/POST	/api/feed/*	Fil d'actualités
GET/POST	/api/messaging/*	Messagerie instantanée
GET	/api/network/*	Réseau de contacts
GET	/api/jobs/*	Offres d'emploi
GET/POST	/api/workspaces/*	Espaces collaboratifs
GET	/api/gamification/*	Badges et classement
GET	/api/notifications/*	Centre de notifications

5.1.7 Données de Test et Jeux de Données

Fichier	Statut	Taille	Contenu
employees.jsonl	✓	1.00 Mo	2 000+ profils d'experts
documents.jsonl	✓	850 Ko	Documents techniques
technologies.jsonl	✓	3.05 Mo	Base technologique complète
projects.jsonl	✓	122 Ko	Projets d'entreprise
companies.jsonl	✓	34 Ko	Entreprises partenaires
skills.jsonl	✓	7.9 Ko	Référentiel de compétences
generate_data.py	✓	20.1 Ko	Générateur configurable (small/medium/large)
Total :		~5.1 Mo	7 fichiers

6. Phase 3 — Développement Frontend

6.1 État : Complète (100%)

L'interface utilisateur React est entièrement développée avec **18 pages** (7 068 lignes de code TSX), un système de thème MUI personnalisé, le graphe 3D interactif, et une architecture complète de services. L'ensemble du frontend comprend **27 fichiers source** pour un total de **~516 Ko**.

6.1.1 Pages de l'Application (18 pages — 7 068 lignes)

Page	St.	Taille	Fonctionnalité clé
Login.tsx	✓	9.4 Ko	Authentification JWT
Register.tsx	✓	10.0 Ko	Inscription sécurisée avec validation
Dashboard.tsx	✓	16.7 Ko	Tableau de bord principal (KPIs, graphiques)
ExpertSearch.tsx	✓	33.3 Ko	Recherche multicritères d'experts
Profile.tsx	✓	24.4 Ko	Profil dynamique de l'employé
GraphVisualization.tsx	✓	26.8 Ko	Graphe 3D (react-force-graph-3d)
AIInsights.tsx	✓	41.5 Ko	Assistant IA Veda + analyses prédictives
SkillEvolution.tsx	✓	27.1 Ko	Tendances d'évolution des compétences
SkillMap.tsx	✓	25.1 Ko	Cartographie visuelle des compétences
DocumentSearch.tsx	✓	16.4 Ko	Recherche et classification de documents
Feed.tsx	✓	28.9 Ko	Fil d'actualité social
Messaging.tsx	✓	16.3 Ko	Messagerie instantanée
TeamBuilder.tsx	✓	23.4 Ko	Composition d'équipes optimisées par IA
TeamWorkspace.tsx	✓	39.6 Ko	Espace de travail collaboratif
LearningPath.tsx	✓	86.2 Ko	Parcours d'apprentissage personnalisé
Jobs.tsx	✓	14.2 Ko	Offres d'emploi et matching
MyNetwork.tsx	✓	13.6 Ko	Réseau de contacts professionnel
Notifications.tsx	✓	8.6 Ko	Centre de notifications

6.1.2 Architecture Frontend

Module	Statut	Description
services/api.ts	✓	Client API centralisé (Axios, 14.2 Ko)
contexts/AuthContext.tsx	✓	Contexte d'authentification React (2.2 Ko)
theme/theme.ts	✓	Thème MUI personnalisé dark mode (1.5 Ko)
components/AIChatbot.tsx	✓	Composant chatbot flottant (18.0 Ko)
App.tsx	✓	Routage principal React Router v6 (25.1 Ko)
main.tsx	✓	Point d'entrée React (230 o)

6.1.3 Stack Technique Frontend

Catégorie	Technologie	Version
Framework	React + TypeScript	v19.2
Build Tool	Vite	v7.2
Bibliothèque UI	Material UI (MUI)	v7.3
Graphe 3D	react-force-graph-3d + Three.js	v1.29 + v0.182
Graphiques	Recharts	v3.7
Routage	React Router	v7.10
HTTP Client	Axios	v1.13
Serveur de production	Nginx Alpine	—

7. Phase 4 — Intégration & Tests

7.1 État : En cours (80%)

La suite de tests a été significativement développée avec **6 fichiers de test**, **21 classes de test** et **~64 cas de test** couvrant la majorité des endpoints API. La couverture a progressé de 60% à 80% par rapport au dernier rapport.

7.1.1 Infrastructure de Tests

Fichier de test	Statut	Lignes	Couverture
conftest.py	✓	38	Fixtures partagées (client, JWT, auth_headers)
test_auth.py	✓	174	Inscription, Login, /me, validation, erreurs
test_api.py	✓	192	IA, Messaging, Gamification, Dashboard, Health
test_experts.py	✓	62	Recherche experts, filtres, localisations
test_documents.py	✓	46	Recherche documents, types, filtres
test_dashboard.py	✓	54	Stats, top-skills, distribution, silos
test_routes.py	✓	123	Feed, Network, Jobs, Workspace, Learning, Graph
Total :		554	~64 cas de test, 21 classes

7.1.2 Détail de la Couverture par Module

Module / Router	Tests écrits	Nb tests	Statut
Authentification (auth)	✓	13	Inscription + Login + /me + erreurs
Intelligence Artificielle (ai)	✓	5	Chatbot + Recommend + Stats + Tr
Recherche d'experts (experts)	✓	7	Filtres + pagination + localisation
Documents (documents)	✓	5	Recherche + filtres + types
Dashboard (dashboard)	✓	12	Stats + skills + départements + proje
Messagerie (messaging)	✓	3	Conversations + envoi + messages
Gamification	✓	2	Badges + leaderboard
Notifications	✓	2	Liste + marquer comme lues
Feed (fil d'actualité)	✓	2	Liste posts + création
Network (réseau)	✓	4	Suggestions + connexions + stats
Jobs (emploi)	✓	3	Liste + recommandés + filtres
Workspaces	✓	2	Liste + création
Learning (apprentissage)	✓	2	Skills disponibles + génération parcou
Graphe	✓	2	Nœuds + statistiques
Health Check	✓	2	Root endpoint + /health
BigData / Spark	▲	0	Non couvert
Kafka Simulator	▲	0	Non couvert
SkillMap	▲	0	Non couvert
Learning Resources	▲	0	Non couvert
Tests unitaires ML (algorithmes)	✗	0	Non couvert
Tests Frontend (React)	✗	0	Non configuré
Tests de Performance	✗	0	Non démarrés

7.1.3 Actions Restantes pour la Phase 4

1. **Tests unitaires ML (priorité haute)** : Valider les algorithmes PageRank, Recommender, Classifier avec des jeux de données de référence.
2. **Tests des routers restants** : Couvrir BigData, Kafka Simulator, SkillMap, Learning Resources.
3. **Tests de performance** : Mesurer les temps de réponse des requêtes Neo4j avec >2 000 nœuds.

8. Phase 5 — Déploiement & Livraison

8.1 État : Complète (100%)

L'ensemble de l'infrastructure de déploiement est opérationnel et testé.

8.1.1 Infrastructure Docker

Composant	Statut	Description
docker-compose.yml	✓	Orchestration 3 services + volume Neo4j
backend/Dockerfile	✓	Python 3.11 + FastAPI + Uvicorn (630 o)
frontend/Dockerfile	✓	Node 20 build → Nginx Alpine (544 o)
frontend/nginx.conf	✓	Proxy API + SPA routing (742 o)

```

docker-compose.yml
|- neo4j:5-community                                (ports 7474, 7687)
|   +- Volume persistant : neo4j_data
|   +- Plugin APOC activ  
|   +- Healthcheck int  gr   (15s interval)
|- backend                                          (port 8000)
|   |- FastAPI + Uvicorn (auto-reload)
|   |- Montage : ./data -> /app/data
|   +- D  pend de : neo4j (service_healthy)
+- frontend                                        (port 3000 -> 80)
|- Build multi-stage : Node 20 -> Nginx Alpine
|- Routage SPA + Proxy /api/*
+- D  pend de : backend

```

8.1.2 Documentation du Projet

Document	Statut	Taille	Contenu
README.md	✓	10.1 Ko	Guide complet (Quick Start, API, Data Model)
ARCHITECTURE.md	✓	4.1 Ko	Architecture syst��me (Mermaid diagrams)
PROJECT_ARCHITECTURE.md	✓	43.5 Ko	Documentation d��taill��e compl��te
.gitignore	✓	444 o	Exclusions Git
.github/	✓	—	Configuration GitHub

9. Mod  le de Donn  es Neo4j

Le mod  le de donn  es repose sur une base de donn  es orient  e graphe Neo4j 5 Community avec le plugin APOC :

9.1 N  uds

N��ud	Propri��t��s principales
:Expert	id, name, email, department, role, location, experience_years, expertise_level
:Skill	id, name, category, level, demand
:Document	id, title, type, topic, content, author, date, views, rating
:Project	id, name, domain, tech_stack, status, budget, priority

9.2 Relations

Relation	Direction
HAS_SKILL	Expert → Skill
WORKS_ON	Expert → Project
AUTHORED	Expert → Document
KNOWS	Expert → Expert
REQUIRES_SKILL	Project → Skill
COVERS_TOPIC	Document → Topic

10. Statut du Projet Comparé au Diagramme de Gantt

Le tableau ci-dessous met en exergue l'alignement entre le calendrier prévisionnel et la réalité de l'exécution.

Phase du Planning	Période Prévvue	Statut Réel	Écart / Délais
1. Analyse & Conception	S1–S2	Terminée à 100%	✓ Respecté
2. Dév. Backend	S3–S5	Terminé à 100%	✓ Respecté
3. Dév. Frontend	S4–S6	Terminé à 100%	✓ Respecté
4. Intégration & Tests	S6–S7	En cours (80%)	▲ Léger retard
5. Déploiement	S8	Terminé à 100%	↑ En avance

10.1 Analyse des Écarts

- **Parallélisation réussie (Phases 2 & 3)** : Le chevauchement planifié entre Backend (S3–S5) et Frontend (S4–S6) a été très efficace. Les contrats d'API FastAPI fixés dès la S3 ont permis un développement frontend sans blocage.
- **Avance sur le déploiement (Phase 5)** : La conteneurisation Docker a été anticipée, expliquant son statut à 100% alors que les tests ne sont pas encore finalisés.
- **Progression des tests (Phase 4)** : Depuis le dernier rapport, la couverture de tests est passée de **60% à 80%** grâce à l'ajout de 6 fichiers de test couvrant 15 des 18 routers API. Les 20% restants concernent les tests unitaires des algorithmes ML et les routers BigData/Kafka.

11. Métriques Quantitatives du Projet

11.1 Tableau de bord récapitulatif

Catégorie	Prévu	Réalisé	Avancement	Statut
Documents de conception (LaTeX)	6	6	100%	✓
Livrables PDF compilés	6	6	100%	✓
Diagrammes UML (PNG)	7	7	100%	✓
Maquettes UI/UX	5	5	100%	✓
Modules API (routers)	18	18	100%	✓
Modules ML/IA	9	9	100%	✓
Modules Spark (Big Data)	7	7	100%	✓
Pages Frontend	18	18	100%	✓
Fichiers de données (JSONL)	6	6	100%	✓
Fichiers Docker	4	4	100%	✓
Fichiers de test backend	–	6	80%	▲

11.2 Métriques de Code Source

Composant	Fichiers	Lignes de code	Taille
Backend Python			
Routers API (18 modules)	18	5 532	284 Ko
Modules ML/IA (9 modules)	9	1 835	85 Ko
Infrastructure Core	7	~300	19 Ko
Spark / Big Data	6	~800	55 Ko
Tests	7	554	27 Ko
Sous-total Backend	40+	~9 000	~386 Ko
Frontend TypeScript/React			
Pages (18 pages)	18	7 068	462 Ko
Services + Contexts + Theme	3	~300	18 Ko
Composants	1	~350	18 Ko
App + Config	5	~500	27 Ko
Sous-total Frontend	27	~8 200	~516 Ko
Données			
Fichiers JSONL	6	—	5.07 Mo
Générateur de données	1	~500	20 Ko
Documentation			
Documents LaTeX	7	~4 000	189 Ko
Markdown (README, ARCH.)	3	~550	58 Ko
Diagrammes (PNG)	7	—	1.1 Mo
Livrables PDF	6	—	4.7 Mo
TOTAL PROJET	~100+	~17 200+	~12 Mo

11.3 Stack Technologique Complète

Domaine	Technologies
Frontend	React 19, TypeScript 5.9, MUI 7, Vite 7, Recharts 3, Three.js
Backend	FastAPI, Pydantic 2, Uvicorn, Python 3.11
Base de données	Neo4j 5 Community + APOC
ML / IA	scikit-learn, NumPy, Pandas, NLTK, TF-IDF, PageRank
IA Générative	Groq API (LLM), Graph RAG
Big Data	PySpark 3.5+, simulation Kafka
Authentification	JWT (python-jose), bcrypt, OAuth2, RBAC
Temps réel	WebSocket (FastAPI), Manager de connexions
Déploiement	Docker, Docker Compose, Nginx Alpine
Tests	Pytest, FastAPI TestClient, unittest.mock
Versioning	Git, GitHub

12. Plan d'Action Restant

Pour atteindre les **100%** de complétion, les actions suivantes sont nécessaires :

Priorité	Action	Durée estimée	Responsable
P1	Tests unitaires des algorithmes ML	1–2 jours	Dev 1
P2	Tests des routers restants (BigData, Kafka, SkillMap)	1 jour	Dev 1
P2	Tests de scénarios bout-en-bout	1 jour	Dev 1 + Dev 2
P3	Tests de performance (Neo4j + Graphe 3D)	1 jour	Dev 2
P3	Correction des bugs éventuels	1 jour	Dev 1 + Dev 2

Estimation de finalisation : Le projet peut atteindre 100% de complétion en **3 à 5 jours ouvrés** supplémentaires, en se concentrant sur la validation et les tests des modules restants.

13. Conclusion

Le projet **Nexora** affiche un taux d'avancement global de **95%**. L'ensemble des composants fonctionnels sont **intégralement développés et opérationnels** :

- **Conception** : 6 documents LaTeX + 6 PDFs + 7 diagrammes UML + 5 maquettes UI
- **Backend** : 18 routers API (5 532 lignes) + 9 modules ML/IA (1 835 lignes) + 7 modules Spark

- **Frontend** : 18 pages React (7 068 lignes) + graphe 3D interactif + thème MUI
- **Données** : 6 fichiers JSONL (~5 Mo, 2000+ experts) + générateur configurable
- **Déploiement** : Docker Compose 3 services + Nginx + healthchecks
- **Tests** : 6 fichiers de test, 64 cas de test couvrant 15/18 routers

Le projet totalise plus de **17 200 lignes de code source** réparties dans **~100 fichiers**, utilisant un écosystème de **11+ technologies** couvrant le développement full-stack, l'IA/ML, le Big Data et le DevOps.

Le seul axe de travail restant concerne la complétion de la **Phase 4 (Tests)**, qui nécessite principalement les tests unitaires des algorithmes ML et la couverture des 3 derniers routers. Le planning initial de 8 semaines a été respecté dans sa globalité, avec une exécution parallèle réussie des phases Backend et Frontend (semaines 4 à 6).